# 하네스 엔지니어링: 목차

**모델을 뺀 모든 것을 설계해, 에이전트가 잘하고 사고 없이 일하게 만듦**

## 이 모듈에서 배우는 것

- 하네스가 무엇인지, 왜 결과를 바꾸는지
- 하네스 4층으로 장치를 나눠 보기
- 보안은 필터가 아닌 구조로 설계하기
- 최소 하네스로 시작해 실패로 키우기


### 1 하네스 엔지니어링이란

- 같은 모델 다른 결과
- 정의
- 4층 지도
- 한계

### 2 1층 미리 안내

- 지침 파일
- 도구 계약
- 컨텍스트
- 상태 저장

### 3 2층 결과 확인

- 센서 두 종류
- 실행 검증
- 완료 기준
- 전파 차단

### 4 3층 실행 중 통제

- 리스크 지도
- 비용
- 안정성
- 보안
- MAS 위험

### 5 4층 관측과 개선

- 트레이스
- 평가
- 조향 루프
- 다이어트

### 6 적용 절차와 실습

- 6단계
- 최소 하네스
- 사례
- 실습


상세 보기 →

무단전재 및 배포 금지

# 하네스 엔지니어링: 같은 모델, 다른 결과

**모델은 그대로 두고 하네스만 바꿔도 결과가 크게 달라짐**

**LangChain 코딩 에이전트**

## 52.8 → 66.5

Terminal Bench 2.0, 모델 고정(gpt-5.2-codex)

**하네스 공개 촉구 논문의 인용 사례**

## 69.7% → 77.0%

Terminal-Bench 2 pass@1, 모델 고정

### 무엇이 달라졌나

**1 무엇을 보여 주나**
지침, 환경 정보, 컨텍스트

**2 무엇을 할 수 있나**
도구, 권한, 실행 환경

**3 언제 확인하고 멈추나**
검증, 종료 조건

> ***같은 모델이라도 어떤 환경에서 일하느냐가 결과를 가름***

상세 보기 →

출처: LangChain, Improving Deep Agents with harness engineering(2026-02-17) / Zhang 외, arXiv 2605.23950(2026-05)

무단전재 및 배포 금지

# 하네스 엔지니어링: Agent = Model + Harness

**하네스는 에이전트에서 모델을 뺀 모든 것**

## 에이전트 = 모델 + 하네스




### 잘하게 만들기

지침, 도구, 컨텍스트, 검증으로 원하는 결과를 내게 함

### 사고 안 나게 막기

종료 조건, 권한, 샌드박스, 사람 승인으로 피해를 막음

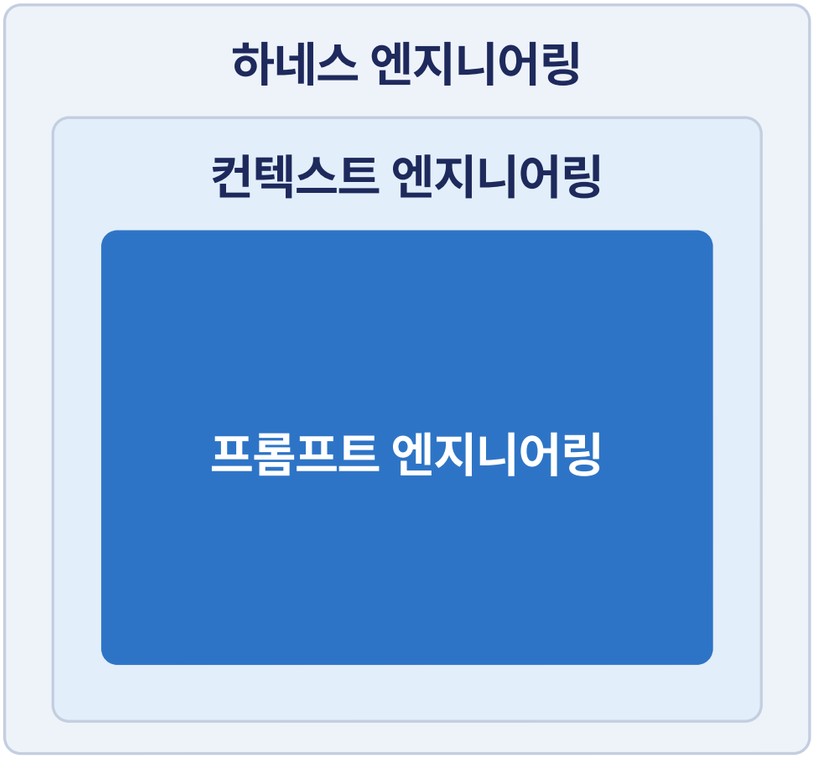
상세 보기 →

출처: Böckeler, martinfowler.com(2026-04-02) / LangChain, The Anatomy of an Agent Harness(2026-03-10) / Hugging Face(2026-05-25)

무단전재 및 배포 금지

# 하네스 엔지니어링: 프롬프트, 컨텍스트, 하네스

**관심이 "무엇을 말할까"에서 "어떤 환경에서 일하게 할까"로 넓어짐**




| 단계 | 묻는 질문 | 다루는 범위 | 예 |
|---|---|---|---|
| **프롬프트 엔지니어링** | 어떻게 지시할까 | 지시문 한 번 | 역할, 형식, 예시 |
| **컨텍스트 엔지니어링** | 무엇을 보여 줄까 | 모델에 들어가는 정보 전체 | 검색 결과, 이력, 도구 결과 |
| **하네스 엔지니어링** | 어떤 환경에서 일하게 할까 | 실행 환경, 도구, 검증, 권한, 개선 | 테스트, 승인, 트레이스 |

> ***다음 단계가 앞 단계를 버리는 것이 아니라 안에 품음***

상세 보기 →

출처: Anthropic, Effective context engineering for AI agents(2025-09-29) / 디지털타임스(2026-05-18)

무단전재 및 배포 금지

# 하네스 엔지니어링: 하네스 4층 지도

**미리 안내하고, 결과를 확인하고, 실행 중에 막고, 기록을 보고 고침**

| | | |
|---|---|---|
| **1층**<br>**미리 안내(가이드)** | **시작 전에 방향을 줌**<br>지침 파일, 도구 계약, 컨텍스트, 상태 저장 | 2장 |
| **2층**<br>**결과 확인(센서)** | **한 일을 확인하고 되돌림**<br>테스트, 스키마 검증, 평가 에이전트 | 3장 |
| **3층**<br>**실행 중 통제(가드레일)** | **사고를 실행 중에 막음**<br>반복 상한, 예산, 권한, 사람 승인 | 4장 |
| **4층**<br>**관측과 개선** | **기록을 보고 하네스를 고침**<br>트레이스, 평가셋, 조향 루프 | 5장 |

조향 루프

## 통제는 코드가 집행

- 권한과 상한은 모델 바깥의 규칙이 검사함
- 모델이 지시를 우회하려 해도 넘을 수 없음
- 허용 여부를 LLM에 묻지 않음

상세 보기 →

출처: Böckeler, martinfowler.com(2026-04-02) / The New Stack(2026-08-30) / KISA, 인공지능(AI) 보안 안내서 부록(2025-12)

무단전재 및 배포 금지

# 하네스 엔지니어링: 하네스의 한계와 반론

**하네스는 모델이 좋아지면 낡음. 더하는 만큼 빼는 일도 설계함**

## 반론과 한계

### 하네스 차이가 작다는 결과도 있음

같은 모델을 여러 하네스에서 돌려도 차이가 오차 범위라는 벤치마크가 소개됨

### 지침 파일이 늘 돕지는 않음

컨텍스트 파일은 대체로 성공률을 높이지 못하고 비용만 20% 넘게 늘림

### 하네스는 낡음

새 모델이 서두르는 버릇을 고치자 컨텍스트 리셋 장치가 필요 없어짐

## 그래서 이렇게

1. 가볍게 시작함
2. 효과는 평가로 확인한 뒤 남김
3. 모델이 바뀌면 하나씩 빼 봄(5-5)

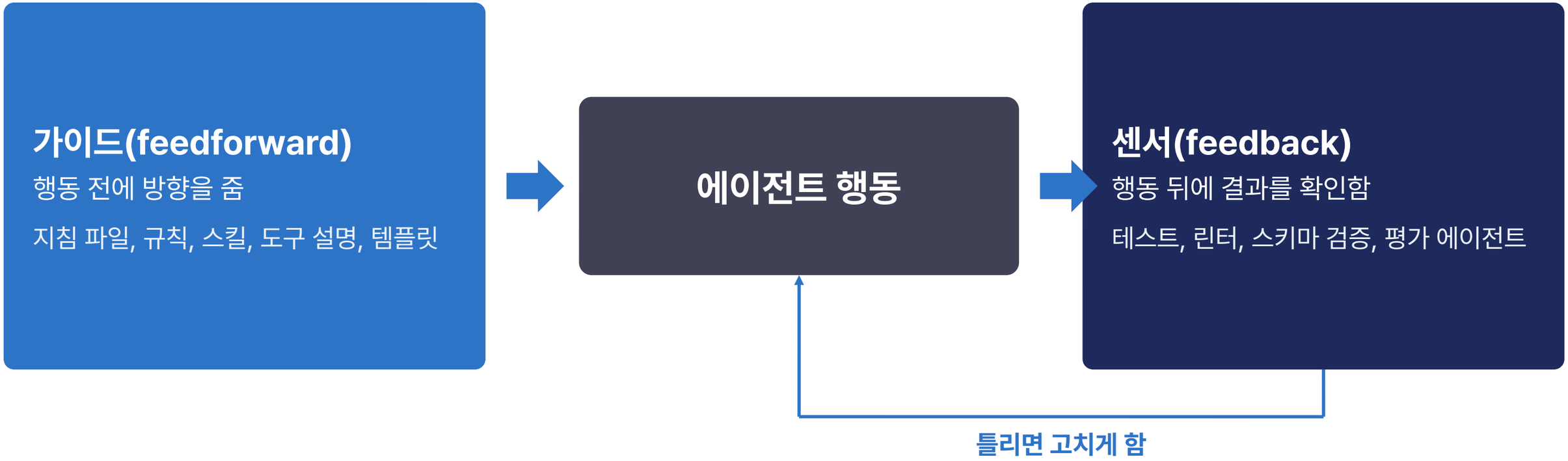
상세 보기 →

출처: Latent Space(2026-03-05) / ETH Zurich 외, arXiv 2602.11988 v3 / Anthropic, Harness design for long-running apps(2026-03-24)

무단전재 및 배포 금지

# 하네스 엔지니어링: 가이드, 미리 안내

**실수를 미리 막는 안내(가이드)와 실수를 잡아내는 검사(센서)가 짝을 이룸**




가이드만 있으면 지켰는지 모르고, 센서만 있으면 같은 실수를 반복함. 둘을 함께 설계함

상세 보기 →

출처: Böckeler, Harness engineering for coding agent users(martinfowler.com, 2026-04-02)

무단전재 및 배포 금지

# 하네스 엔지니어링: 지침 파일은 목차처럼

**지침 파일은 짧은 지도로 두고, 코드만 봐서는 알 수 없는 규칙만 적음**

## 실패한 방식: 거대한 지침 파일 하나

- 컨텍스트를 잡아먹어 정작 할 일을 놓침
- 모두 중요하면 아무것도 중요하지 않음
- 금방 낡고, 무엇이 맞는지 알 수 없음
- 기계로 검사하기 어려움

## 바꾼 방식: 목차 + 문서 폴더

**AGENTS.md 약 100줄 = 목차**

↓

**docs/ 설계 문서, 실행 계획, 규칙 → 자세한 내용**

- 린터와 CI가 문서의 연결과 최신 상태를 검사
- 정기 문서 정리 에이전트가 낡은 문서를 고침

**연구 결과** 저장소 개요 설명은 도움이 안 됨. 표준과 다른 작업 방식을 알릴 때만 쓸모 있음(ETH, 2026)

상세 보기 →

출처: OpenAI, Harness engineering: leveraging Codex in an agent-first world(2026-02-11) / arXiv 2602.11988

무단전재 및 배포 금지

# 하네스 엔지니어링: 도구 계약 설계

**도구는 좁은 기능만 주고, 입력과 오류를 분명히 약속함**

| 관점 | 피할 것 | 권장 |
|---|---|---|
| **기능 범위** | 셸 실행처럼 무엇이든 되는 도구 | 파일 쓰기만, 메일 읽기만 같은 좁은 도구 |
| **권한** | 읽기, 쓰기, 삭제를 한 번에 | 필요한 권한만(읽기 전용 범위 등) |
| **입력** | 자유 문자열 | 형식이 정해진 매개변수 |
| **오류 응답** | 내부 오류를 그대로 | 무엇이 틀렸고 어떻게 고칠지 |
| **설명** | 이름만 | 언제 쓰는지, 예시, 주의점 |

**사례**

## 메일 요약 에이전트

- 메일 요약용 플러그인에 '보내기' 기능까지 들어 있음
- 악성 메일 한 통이 에이전트를 속여 스팸 발송
- 대책: 읽기 전용 도구, 읽기 전용 권한, 발송은 사람 확인

상세 보기 →

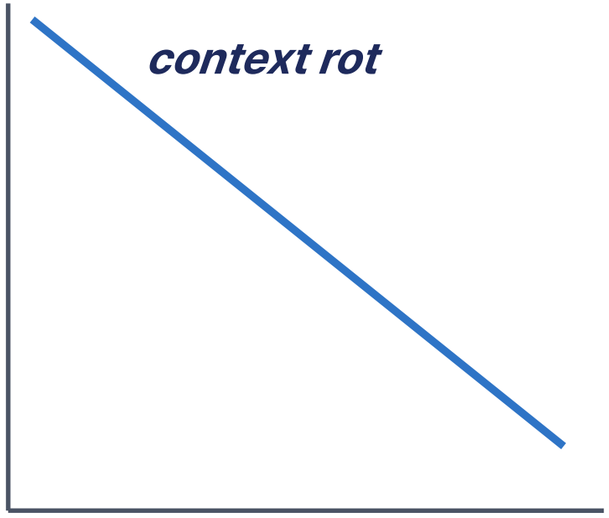
출처: Anthropic, Building effective agents(2024-12-19) / KISA, 인공지능(AI) 보안 안내서 부록 158 ~ 159쪽(2025-12)

무단전재 및 배포 금지

# 하네스 엔지니어링: 컨텍스트 관리

**컨텍스트가 길어지면 비용도 오르고 정확도도 떨어짐**

기억 정확도




개념 그림

| 기법 | 하는 일 | 주의 |
|---|---|---|
| **컴팩션** | 한계에 가까워지면 대화를 요약해 이어 감 | 중요한 내용이 빠질 수 있음 |
| **노트 남기기** | 진행 상황을 파일에 적고 필요할 때 다시 읽음 | 노트 형식을 정해 둠 |
| **도구 결과 지우기** | 다 쓴 도구 결과를 컨텍스트에서 뺌 | 다시 필요하면 다시 조회 |
| **서브에이전트 분리** | 깊은 조사는 서브에이전트가 하고 요약만 돌려줌 | 요약의 정확도를 확인 |
| **필요할 때만 검색** | 처음에 다 넣지 않고 파일 경로 같은 단서만 둠 | 검색 도구의 품질이 중요 |

프롬프트 캐싱은 고정 지침을 앞에 두어 같은 내용의 반복 비용을 줄임(앞선 교재 143쪽)

상세 보기 →

출처: Anthropic, Effective context engineering for AI agents(2025-09-29) / MAST, arXiv 2503.13657

무단전재 및 배포 금지

# 하네스 엔지니어링: 상태 저장과 이어 하기

**컨텍스트 창이 바뀌어도 작업이 끊기지 않도록 상태를 바깥에 남김**

**초기화 에이전트(1회)**
실행 환경 준비, 기능 목록 작성

**코딩 에이전트(반복)**
세션마다 기능 하나만 구현하고 확인

**공유 산출물**

- init.sh 실행 준비
- 진행 기록 파일
- 기능 목록 JSON(통과 여부만 수정)
- git 커밋

## MAS로 옮기면

- LangGraph 체크포인터로 단계마다 상태 저장
- 세션 기록은 덧붙이기만 하는 로그로
- 실행 공간(샌드박스)과 기록을 분리해 하나가 죽어도 이어 감

상세 보기 →

출처: Anthropic, Effective harnesses for long-running agents(2025-11-26) / Scaling Managed Agents(2026-04-08)

무단전재 및 배포 금지

# 하네스 엔지니어링: 센서의 두 종류

**코드로 판정할 수 있는 검사를 먼저, LLM 심사는 보조로 씀**

| 구분 | 코드로 판정(연산형) | LLM이 판단(추론형) |
|---|---|---|
| **예** | 테스트, 린터, 타입 검사, 스키마 검증 | 평가 에이전트, LLM 심사 |
| **속도와 비용** | 빠르고 쌈 | 느리고 비쌈 |
| **결과** | 매번 같음 | 실행마다 다를 수 있음 |
| **잘 잡는 것** | 형식, 규칙, 동작 | 의미, 품질, 설계 취향 |

> ***둘을 함께 쓰되, 싸고 확실한 코드 검사를 먼저 돌림***




상세 보기 →

출처: Böckeler, Harness engineering for coding agent users(martinfowler.com, 2026-04-02)

무단전재 및 배포 금지

# 하네스 엔지니어링: 실제로 돌려 보고 확인

**에이전트의 "다 했다"를 믿지 않고, 실행 결과로 확인함**

### 테스트 실행

기능이 실제로 동작하는지 확인

### 린터와 구조 테스트

계층 규칙, 이름 규칙, 파일 크기 같은 규칙을 기계로 강제

### 끝에서 끝까지 확인

브라우저로 사람처럼 화면을 눌러 봄

### 고치는 방법을 담은 오류 메시지

예시

오류: domain 계층이 ui 계층을 불러옴

고치는 법: 공용 타입은 types/ 폴더로 옮기고 그곳에서 불러올 것

상세 보기 →

출처: OpenAI, Harness engineering(2026-02-11) / Anthropic, Effective harnesses for long-running agents(2025-11-26)

무단전재 및 배포 금지

# 하네스 엔지니어링: 완료 기준을 먼저 합의

**만드는 에이전트와 평가하는 에이전트를 나누고, 작업 전에 "끝"의 기준을 정함**




**스프린트 계약**

코드를 쓰기 전에 "다 됨"의 모습을 둘이 합의

**왜 나누나**

1. 만든 쪽이 스스로 평가하면 후해짐
2. 평가자는 채점 기준과 예시로 따로 보정
3. 기준이 먼저 있어야 "다 했다"를 판정 가능

상세 보기 →

출처: Anthropic, Harness design for long-running apps(2026-03-24)

무단전재 및 배포 금지

# 하네스 엔지니어링: 할루시네이션 전파 차단

**검증을 통과한 결과만 다음 에이전트로 넘김**




| 순서 | 해결책 | 설명 |
|---|---|---|
| 1 | 스키마 검증 | 정해진 JSON, Pydantic 형식과 필수 필드를 만족하는지 확인 |
| 2 | 근거 대조 | DB, 문서, API, 검색 결과 같은 외부 근거와 대조 |
| 3 | 교차검증 | 다른 모델, 다른 프롬프트, 다른 에이전트로 검토해 일관성 확인 |
| 4 | 실패 시 체인 차단 | 다음 에이전트로 넘기지 않고 재시도하거나 사람 확인으로 올림 |

**주의** **모델이 스스로 낸 확신도(confidence)만 믿지 않음. 출처 유무, 검증 결과, 모델 간 일치를 함께 봄**

상세 보기 →

출처: 앞선 교재 144쪽 / Zhang 외, arXiv 2605.23950(2026-05)

무단전재 및 배포 금지

# 하네스 엔지니어링: 사례, 완료 전 자기 검증

**하네스만 고쳐 점수를 52.8에서 66.5로 올린 장치들**




| 장치 | 하는 일 |
|---|---|
| **완료 전 점검** | 끝내려는 순간 가로채 과제 명세와 대조하는 검증을 한 번 더 돌리게 함 |
| **환경 정보 주입** | 시작할 때 폴더 구조와 쓸 수 있는 도구를 찾아 미리 알려 줌 |
| **시간 예산 경고** | 남은 시간을 알려 마무리와 검증으로 넘어가게 함 |
| **반복 수정 감지** | 같은 파일을 여러 번 고치면 접근을 다시 생각하라고 알림 |
| **트레이스 분석** | 실패한 실행 기록을 모아 원인을 찾고 하네스를 고침 |

상세 보기 →

출처: LangChain, Improving Deep Agents with harness engineering(2026-02-17)

무단전재 및 배포 금지

# 하네스 엔지니어링: MAS 실행 리스크 지도

**9대 실행 리스크에 협업 실패를 더해 MAS의 사고 지점을 한눈에 봄**

| 비용 | 안정성과 품질 | 보안 | 협업 |
|---|---|---|---|
| **무한 루프**<br>반복 실행으로 비용 폭증 | **시스템 마비**<br>과부하로 중단 | **Prompt Injection**<br>악성 입력으로 의도 탈취 | **역할 위반**<br>맡은 역할을 벗어남 |
| **토큰 누수**<br>불필요한 토큰 사용 | **응답 지연**<br>느린 처리 | **데이터 유출**<br>민감 정보 외부 노출 | **정보 누락 전달**<br>필요한 정보를 넘기지 않음 |
| **Agent 폭주**<br>과다한 병렬 실행 | **할루시네이션 전파**<br>틀린 답이 다음 단계로 확산 | **권한 오남용**<br>과도한 권한 사용 | **결과 무시**<br>다른 에이전트의 결과를 무시 |

참고 분류 **MAST 다중 에이전트 실패 14유형** **OWASP 에이전트 보안 Top 10(ASI01 ~ ASI10)**

> ***반복 상한, 예산 상한, 최소 권한, 고위험 작업 사람 승인은 첫날부터. 나머지는 단계적으로***

상세 보기 →

출처: 앞선 교재 142쪽(교재 고유 분류) / MAST, arXiv 2503.13657 / OWASP Top 10 for Agentic Applications 2026(2025-12)

무단전재 및 배포 금지

# 하네스 엔지니어링: 비용 통제

**비용은 나중에 확인할 일이 아니라 실행 중에 제한할 일**

| 무한 루프 대응 | 토큰 누수 대응 | Agent 폭주 대응 |
| --- | --- | --- |
| • 최대 반복 횟수<br>• 에이전트와 워크플로우 타임아웃 분리<br>• 같은 행동 반복 감지 후 중단<br>• 중단 시 부분 결과와 사유 기록 | • 컨텍스트 관리(2-4)<br>• 프롬프트 캐싱<br>• 요청별 입력, 출력 토큰과 비용 기록 | • 시작 전 예산 한도(비용, 토큰, 호출 수)<br>• 누적 사용량 확인 후 초과 시 차단<br>• 호출 깊이와 동시 실행 수 제한<br>• 이상 시 즉시 중단(Kill-switch) |

> ***중단은 LLM 감독자가 아니라 하네스 코드가 집행함***

상세 보기 →

출처: 앞선 교재 143쪽 / Anthropic, Building effective agents(2024-12-19) / LangChain(2026-02-17)

무단전재 및 배포 금지

# 하네스 엔지니어링: 안정성 통제

**한 곳의 실패가 전체로 번지지 않게 실행 단계에서 막음**

**장애 확산 막기**

| 장치 | 하는 일 |
|---|---|
| **Circuit Breaker** | 연속 실패하는 LLM API나 도구를 잠시 차단 |
| **리소스 격리** | 에이전트별 CPU, 메모리, 실행 환경 분리 |
| **성능 저하 모드** | 일부가 실패해도 핵심 기능만으로 제한된 답 제공 |
| **Backpressure** | 처리량을 넘는 요청은 대기, 제한, 거절 |

**지연 줄이기**

| 장치 | 하는 일 |
|---|---|
| **병렬 실행** | 서로 기다릴 필요 없는 작업을 동시에 |
| **계층적 타임아웃** | 도구, 단계, 에이전트, 워크플로우마다 따로 |
| **캐싱** | 같은 입력의 LLM 답, 검색, 도구 결과 재사용 |
| **스트리밍** | 중간 결과와 진행 상황을 먼저 보여 줌 |

**에이전트에 적용하면** LLM API가 연속 실패하면 다른 모델로 넘김. 도구 서버가 죽으면 그 도구만 끄고 답변 범위가 줄었음을 알림

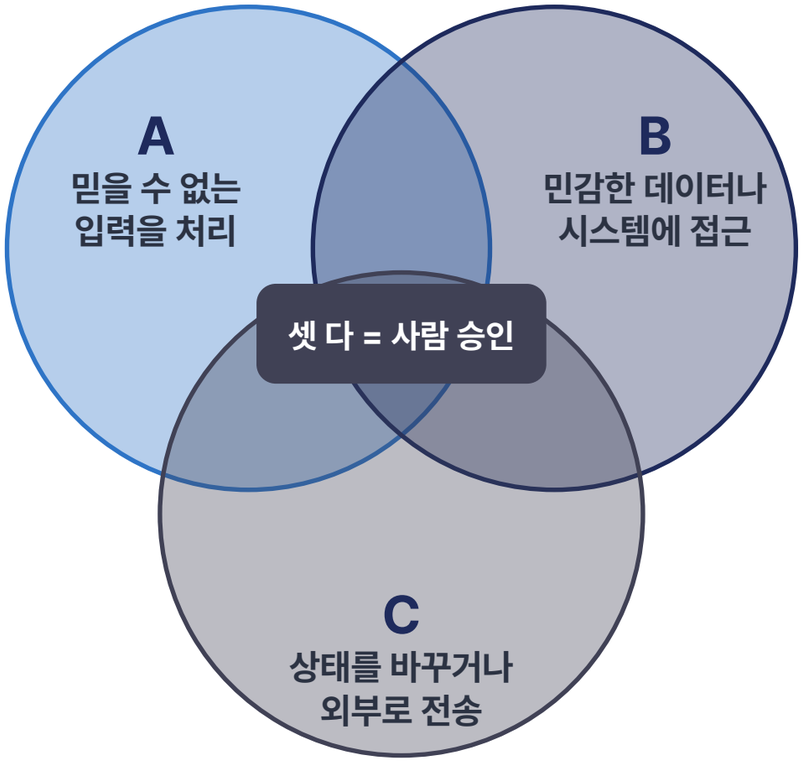
상세 보기 →

출처: 앞선 교재 144쪽 / OWASP Top 10 for Agentic Applications 2026, ASI08 Cascading Failures(2025-12)

무단전재 및 배포 금지

# 하네스 엔지니어링: 보안 1, 구조로 막기

**위험한 능력 세 가지를 한 에이전트에 모두 주지 않음(Rule of Two)**




| 에이전트 예 | A | B | C | 판정 |
|---|---|---|---|---|
| **웹 검색 결과 요약** | O | X | X | 자율 실행 가능 |
| **받은 메일 요약만** | O | O | X | 자율 실행 가능 |
| **받은 메일 읽고 답장 발송** | O | O | O | **사람 승인 필요** |
| **사내 DB 보고서를 정해진 곳에 전송** | X | O | O | 자율 실행 가능 |

사적 데이터, 믿을 수 없는 콘텐츠, 외부 통신이 한곳에 모이면 데이터가 빠져나감(치명적 삼중 조합)

상세 보기 →

출처: Meta AI, Agents Rule of Two(2025-10-31) / Simon Willison, The lethal trifecta(2025-06-16) / CaMeL, arXiv 2503.18813

무단전재 및 배포 금지

# 하네스 엔지니어링: 보안 2, 프롬프트 인젝션 완화

**필터는 우회되므로 보조로 두고, 막는 것이 아니라 피해를 줄이는 쪽으로 설계함**

| 순서 | 해결책 | 설명 |
|---|---|---|
| 1 | 위험 조합 분리 | 믿을 수 없는 입력, 민감 데이터, 외부 전송 중 두 가지까지만 허용(4-4) |
| 2 | 불신 데이터 표시 | 사용자 입력, 웹 문서, 외부 파일, 다른 에이전트 출력을 지시가 아닌 데이터로 다룸 |
| 3 | 샌드박스 | 공격이 성공해도 피해 범위가 좁도록 실행 공간 격리 |
| 4 | 역할 분리 | 공통 지침과 사용자 입력을 나눠 넣어 무시할 가능성을 낮춤(보장 아님) |
| 5 | 입력 필터링(보조) | 알려진 공격 문구를 걸러 냄. 우회될 수 있어 단독으로 쓰지 않음 |

**필터만으로 안 되는 이유**

공개 방어 12개 대부분이 맞춤 공격에 90% 넘게 뚫림(2025-10)

NIST 레드팀 경연: 대상 모델 13종 모두에서 공격 성공(2026-03)

여러 겹 방어 뒤에도 약 1% 성공이 남음 (Anthropic, 2025-11)

상세 보기 →

출처: arXiv 2510.09023 / NIST CAISI(2026-03-23) / Anthropic(2025-11-24) / KISA, 인공지능(AI) 보안 안내서 54쪽(2025-12)

무단전재 및 배포 금지

# 하네스 엔지니어링: 보안 3, 데이터와 권한

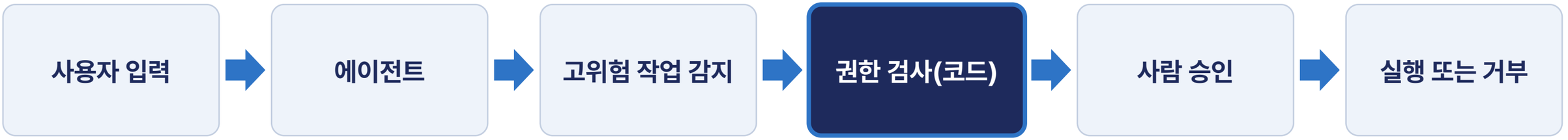
**허용 여부는 LLM이 아니라 연결된 시스템이 검사함**

**데이터 유출 대응**

| 장치 | 하는 일 |
|---|---|
| **DLP 필터** | 출력과 외부 전송에서 개인정보, 계좌, 카드 번호 탐지 |
| **민감 정보 마스킹** | 탐지된 값을 가리거나 지움 |
| **네트워크 허용 목록** | 접근할 수 있는 API, 도메인만 허용 |
| **외부 전송 감사 로그** | 누가, 무엇을, 어디로, 어떤 요청으로 보냈는지 |

**권한 오남용 대응**

| 장치 | 하는 일 |
|---|---|
| **최소 권한** | 에이전트마다 필요한 권한만 |
| **도구 허용 목록** | 허용되지 않은 도구 호출 차단 |
| **연결 시스템에서 검사** | 권한 판단을 LLM이 아닌 연결 시스템이 함 |
| **고위험 작업 사람 승인** | 삭제, 결제, 외부 전송은 승인 뒤 실행 |




상세 보기 →

출처: 앞선 교재 145쪽 / KISA, 인공지능(AI) 보안 안내서 부록 158 ~ 159쪽(2025-12) / OWASP LLM01(2025)

무단전재 및 배포 금지

# 하네스 엔지니어링: MAS만의 위험

**다른 에이전트가 보낸 메시지와 공유 메모리도 믿을 수 없는 입력으로 다룸**

ASI07

## 에이전트 간 통신

무슨 일이 생기나

가짜 메시지나 숨은 지시가 다른 에이전트로 전달됨

대응

- 에이전트마다 신원 인증
- 메시지 서명과 변조 확인
- 허용한 프로토콜 버전만 사용

ASI06

## 메모리와 컨텍스트 오염

무슨 일이 생기나

공유 메모리에 나쁜 내용이 저장되어 계속 쓰임

대응

- 저장 전 내용 검사
- 세션별 메모리 분리와 출처 기록
- 검증 안 된 기억은 만료

ASI08

## 연쇄 실패

무슨 일이 생기나

한 에이전트의 오류가 줄줄이 번짐

대응

- 검증 실패 시 체인 차단(3-4)
- Circuit Breaker(4-3)

> ***에이전트끼리 연결되면 한 곳의 사고가 협력사까지 번질 수 있음(국내 업계 인터뷰)***

상세 보기 →

출처: OWASP Top 10 for Agentic Applications 2026(2025-12) / 지디넷코리아(2026-06-29)

무단전재 및 배포 금지

# 하네스 엔지니어링: 트레이스, 모든 것을 기록

**에이전트가 무엇을 보고 무엇을 했는지 전부 남겨야 원인을 찾음**

예시




## 남길 것

- 단계와 순서
- 도구 이름, 입력, 결과
- 토큰, 비용, 시간
- 검증 결과와 차단 사유
- 사람 승인 기록

트레이스는 감사 로그(4-6), 비용 분석(4-2), 하네스 개선(5-3)의 공통 재료

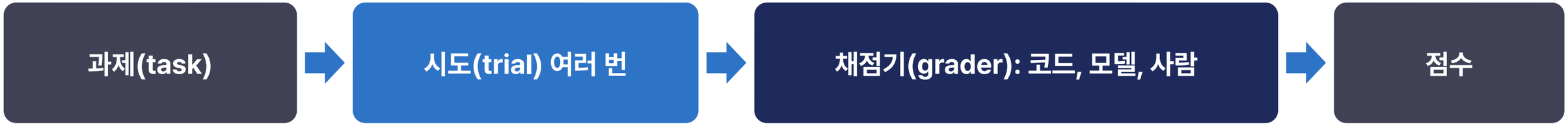
상세 보기 →

출처: Anthropic, Demystifying evals for AI agents(2026-01-09) / LangChain(2026-02-17) / The New Stack(2026-08-30)

무단전재 및 배포 금지

# 하네스 엔지니어링: 평가셋으로 재기

**말이 아니라 결과(환경의 최종 상태)로 채점하고, 하네스와 모델을 함께 평가함**




### 결과로 채점

- 항공권 예약 에이전트가 "예약했습니다"라고 말해도
- 실제 예약 DB에 기록이 있는지로 채점함

| 지표 | 뜻 | 쓰임 |
|---|---|---|
| **pass@k** | k번 중 한 번이라도 성공 | 한 번만 맞으면 되는 일 |
| **pass^k** | k번 모두 성공 | 매번 맞아야 하는 운영 업무 |

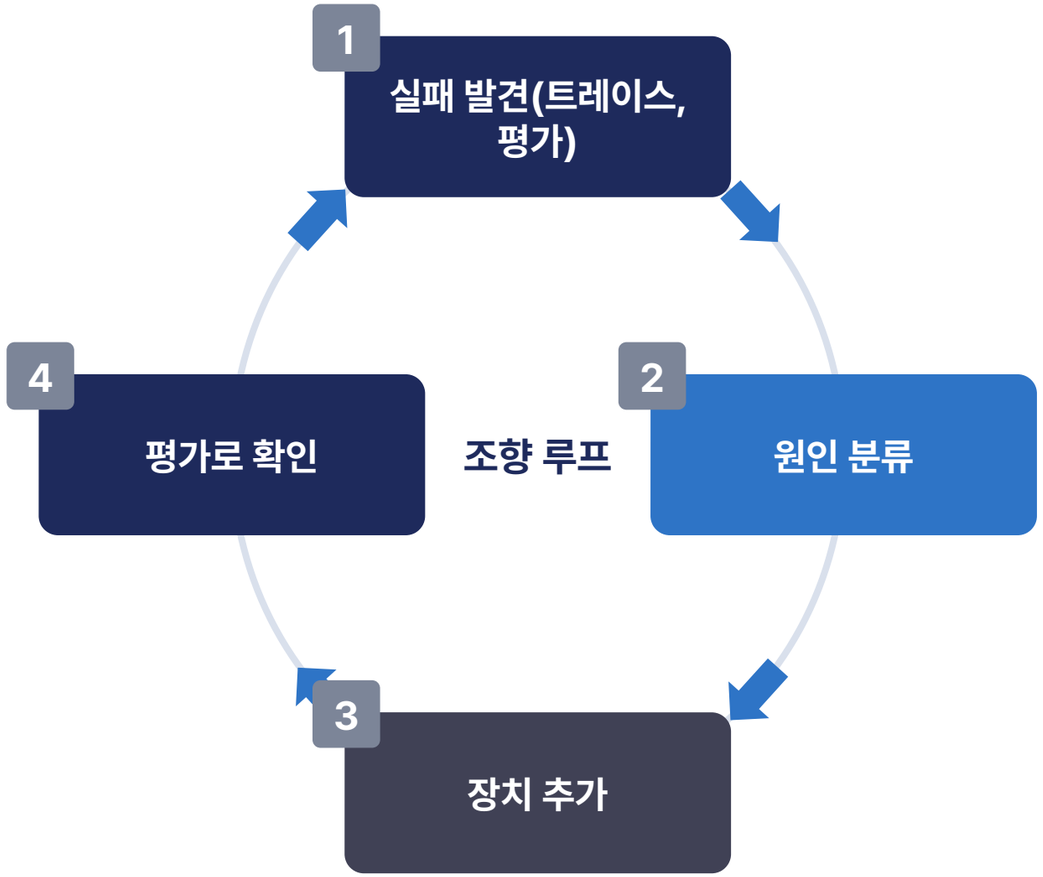
상세 보기 →

출처: Anthropic, Demystifying evals for AI agents(2026-01-09)

무단전재 및 배포 금지

# 하네스 엔지니어링: 조향 루프

**같은 실수가 다시 나지 않도록, 실수할 때마다 장치를 하나 더함**




| 원인 | 추가할 장치 | 층 |
|---|---|---|
| **필요한 정보를 몰랐음** | 지침 한 줄, 문서 연결 | **1층** |
| **결과를 확인하지 않았음** | 테스트, 린터, 스키마 검증 | **2층** |
| **위험한 행동을 했음** | 권한 축소, 사람 승인 | **3층** |
| **원인을 모르겠음** | 트레이스 항목 추가 | **4층** |

상세 보기 →

출처: Mitchell Hashimoto, My AI Adoption Journey(2026-02-05) / Böckeler(2026-04-02) / OpenAI(2026-02-11)

무단전재 및 배포 금지

# 하네스 엔지니어링: 엔트로피 관리

**에이전트가 만든 결과물은 조금씩 어긋나므로 정기적으로 정리함**

## 문제

- 에이전트는 기존 코드의 패턴을 따라 함
- 나쁜 패턴도 그대로 퍼짐

## 황금 원칙

- 공용 도구를 쓰고 직접 만들지 않음
- 데이터 모양을 추측하지 않고 경계에서 검증

## 정기 정리 작업

- 백그라운드 에이전트가 원칙 위반을 찾음
- 작은 수정 요청을 올림
- 대부분 1분 안에 검토

> ***기술 부채는 이자가 높은 빚. 조금씩 자주 갚음***

상세 보기 →

출처: OpenAI, Harness engineering: leveraging Codex in an agent-first world(2026-02-11)

무단전재 및 배포 금지

# 하네스 엔지니어링: 하네스 다이어트

**모델이 바뀌면 장치를 하나씩 빼 보고, 효과 없는 것은 지움**

## 사례

- 이전 모델은 컨텍스트 한계가 가까워지면 서둘러 끝냄
- 그래서 컨텍스트 리셋 장치를 넣음
- 새 모델에서 버릇이 사라져 장치를 뺌

## 절차

1. 장치 목록 작성
2. 하나씩 끄고 평가셋 실행
3. 점수 차이가 없으면 제거

## 연구 동향

- 하네스 코드를 자동으로 탐색하는 Meta-Harness
- 기존 방식보다 7.7점 높고 컨텍스트 토큰은 4분의 1

상세 보기 →

출처: Anthropic, Harness design for long-running apps(2026-03-24) / Meta-Harness, arXiv 2603.28052(2026-03)

무단전재 및 배포 금지

# 하네스 엔지니어링: 적용 6단계

**완료 기준에서 시작해, 실패할 때마다 하네스를 키우고, 모델이 바뀌면 줄임**




상세 보기 →

출처: Anthropic, OpenAI, LangChain, Böckeler 자료를 종합(상세 교재 6-1 참고)

무단전재 및 배포 금지

# 하네스 엔지니어링: 첫날부터 넣을 최소 하네스

**다섯 가지는 처음부터 넣고, 나머지는 실패를 보며 더함**

### 반복 상한
무한 루프를 끊음

3층

### 예산 상한
토큰, 비용, 호출 수 한도

3층

### 최소 권한
도구 허용 목록, 읽기 전용이 기본

3층

### 고위험 작업 사람 승인
삭제, 결제, 외부 전송

3층

### 트레이스
모든 단계를 기록

4층

> ***앞선 교재의 "비용 → 성능 → 보안 순서"를 바꿈. 구조적인 보안 통제는 뒤로 미룰 수 없음***

상세 보기 →

출처: Anthropic, Building effective agents(2024-12-19) / Meta AI, Agents Rule of Two(2025-10-31) / KISA 인공지능(AI) 보안 안내서

무단전재 및 배포 금지

# 하네스 엔지니어링: 도입 순서표

**시스템이 자라는 단계에 맞춰 장치를 더함**

| 층 | 개발 단계 | 운영 전 | 운영 중 |
|---|---|---|---|
| **1층 미리 안내** | 짧은 지침 파일, 도구 설명 | 도구 계약 정리, 컨텍스트 관리 | 낡은 규칙 정리 |
| **2층 결과 확인** | 스키마 검증 | 테스트, 근거 대조, 체인 차단 | 평가 에이전트, 교차검증 |
| **3층 실행 중 통제** | 반복 상한, 예산, 최소 권한, 고위험 승인 | Circuit Breaker, 계층적 타임아웃, DLP, 감사 로그 | Backpressure, 이상 징후 중단 |
| **4층 관측과 개선** | 트레이스 | 평가셋, 회귀 테스트 | 정기 정리, 하네스 다이어트 |

상세 보기 →

출처: 앞선 교재 143 ~ 145쪽과 검토 보고서(2026-11-06)를 바탕으로 교육팀이 정리한 권장안

무단전재 및 배포 금지

# 하네스 엔지니어링: 사례 3가지

**규모는 달라도 "실패를 장치로 바꾼다"는 원리는 같음**

## OpenAI Codex 팀

**상황**

빈 저장소에서 시작해 모든 코드를 Codex가 작성

**핵심 장치**

- 짧은 AGENTS.md와 문서 폴더
- 고치는 법을 담은 맞춤 린터
- 정기 정리 작업

**결과**

**손으로 짤 때의 약 10분의 1 시간에 개발 (팀 추정)**

## Anthropic 장시간 앱 개발

**상황**

몇 시간 동안 자율로 풀스택 앱 제작

**핵심 장치**

- 계획, 생성, 평가 3역할
- 스프린트 계약
- 실제 화면을 눌러 보는 평가

**결과**

**단독 실행보다 품질이 뚜렷이 높음(비용은 20배 이상)**

## 우아한형제들 팀 맞춤 환경

**상황**

팀 규칙에 맞는 AI 개발 환경

**핵심 장치**

- 규칙과 스킬
- 전처리 스크립트로 필요한 정보만 요약 전달

**결과**

**처리 데이터량 평균 96.5% 감소(저자 보고)**

상세 보기 →

출처: OpenAI(2026-02-11) / Anthropic, Harness design for long-running apps(2026-03-24) / 우아한형제들 기술블로그(2026-04-17)

무단전재 및 배포 금지

# 하네스 엔지니어링: 실습 안내

**하네스 없는 문서 Q&A 에이전트에 장치를 하나씩 붙이고 전후를 비교함**

| 단계 | 붙이는 장치 | 층 | 확인 방법 |
|---|---|---|---|
| 0 | 하네스 없는 기본 에이전트 | 없음 | 평가 질문 5개의 점수, 토큰, 시간 기록 |
| 1 | 반복 상한과 예산 상한 | 3층 | 무한 반복을 부르는 질문에서 정해진 횟수에 멈추고 부분 결과 반환 |
| 2 | 짧은 지침과 도구 설명 정리 | 1층 | 같은 질문에서 잘못된 도구 호출 횟수 감소 |
| 3 | 답변 스키마 검증과 근거 대조 | 2층 | 근거 없는 답변이 다음 단계로 넘어가지 않음 |
| 4 | 외부 전송 도구에 사람 승인 | 3층 | 승인 전에는 실행되지 않고 대기 |
| 5 | 트레이스 기록과 평가 재실행 | 4층 | 0단계 대비 점수, 토큰, 실패 유형 비교표 |

## 준비물

팀 저장소 RAG 예제(teacher/rag), LangGraph, 평가 질문 5개

## 결과물

단계별 점수, 토큰, 실패 유형 비교표

상세 보기 →

출처: 교육팀 실습 설계(2026-11)

무단전재 및 배포 금지

# 하네스 엔지니어링: 정리와 점검표

**내 시스템의 하네스를 12문항으로 점검함**

| | | | |
|---|---|---|---|
| 1층 | 지침 파일이 짧은 목차로 되어 있나 | 3층 | 종료 조건과 예산이 코드로 집행되나 |
| 1층 | 도구가 좁은 기능과 분명한 오류 응답을 갖나 | 3층 | 위험한 능력 세 가지가 한 에이전트에 모여 있지 않나 |
| 1층 | 컨텍스트 길이를 관리하나 | 3층 | 고위험 작업에 사람 승인이 있나 |
| 1층 | 작업 상태가 바깥에 저장되나 | 4층 | 모든 단계가 트레이스로 남나 |
| 2층 | 결과를 코드 검사로 먼저 확인하나 | 4층 | 평가셋으로 변경 효과를 재나 |
| 2층 | 검증 실패 결과가 다음 단계로 넘어가지 않나 | 4층 | 실패를 장치로 바꾸고, 낡은 장치를 지우나 |

> ***하네스는 모델을 뺀 모든 것. 잘하게 만들고, 사고 안 나게 막고, 실패에서 계속 고쳐 감***

상세 보기 →

출처: 이 모듈 1 ~ 6장 종합

무단전재 및 배포 금지